PricebookCalc — Google Workspace Migration Plan

A complete plan for rebuilding PricebookCalc using Google Apps Script, Google Sheets, and Google Drive within Google Workspace.

Table of Contents

1. Architecture Mapping
2. Google Sheets Data Model
3. Apps Script Project Structure
4. System Calculator
5. Repair Calculator
6. Reverse Calculator
7. Settings Management
8. Quote Saving & History
9. Authentication & Access Control
10. ServiceTitan Integration
11. Push to ServiceTitan
12. Tax Regions (Locations)
13. Tenant Management
14. Push History & Revert
15. PSI Mode Toggle
16. Commission & Labor Modes
17. UI Implementation with HTML Service
18. Step-by-Step Setup Instructions
19. Limitations & Workarounds
20. Appendix: Code Patterns

---

1. Architecture Mapping

Current Stack → Google Workspace Equivalent

Current (Replit)	Google Workspace Equivalent
React frontend	Apps Script HTML Service (HTML/CSS/JS)
Express.js backend	Apps Script `doGet()` / `doPost()` web app
PostgreSQL database	Google Sheets (one spreadsheet, multiple tabs)
Drizzle ORM	SpreadsheetApp / Sheets API
REST API endpoints	Apps Script `google.script.run` (client→server calls)
TanStack Query (caching)	Client-side JS caching or direct calls
Tailwind CSS	Standard CSS (or include Tailwind CDN)
Vite dev server	Apps Script editor / clasp CLI

Current (Replit)	Google Workspace Equivalent
Environment variables	Script Properties (`PropertiesService.getScriptProperties()`)
OAuth tokens (ServiceTitan)	Cache Service (`CacheService.getScriptCache()`)
Server-Sent Events (push progress)	Polling via `google.script.run`
localStorage (per-device settings)	Client-side `localStorage` (same approach)

Key Architecture Differences

1. **No separate frontend/backend**: Apps Script serves HTML pages and handles server-side logic in a single project.
2. **No real-time updates**: Replace SSE progress tracking with polling (call server function every few seconds during push operations).
3. **No REST API**: Client-side JavaScript calls server functions directly via `google.script.run.functionName()`.
4. **Google Sheets as database**: Each database table becomes a sheet tab. Read/write operations use `SpreadsheetApp`.
5. **Deployment**: Publish as a Google Apps Script web app (accessible via URL with Google account authentication).

---

2. Google Sheets Data Model

Create a single Google Spreadsheet with the following sheet tabs. Each tab replaces a PostgreSQL table.

Sheet: `Settings`

Single row (row 2) with headers in row 1. Auto-create with defaults if empty.

Column	Header	Default Value	Notes
A	id	1	Always 1 (singleton)
B	targetMePercent	23.00	
C	salesTaxPercent	6.35	
D	laborMode	percent	“hours” or “percent”
E	laborHourlyRate	0.00	
F	laborPercent	7.00	
G	laborBurdenPercent	20.00	
H	commissionPercent	10.00	
I	commissionBurdenPercent	20.00	
J	financingFeePercent	6.00	
K	grossMarginMinPercent	52.00	
L	serviceTitanPassword	ZephyrHome160	

Column	Header	Default Value	Notes
M	adminPin	7437	

Sheet: `Quotes`

Each row is a saved calculator snapshot.

Column	Header	Notes
A	id	Auto-increment (max existing + 1)
B	title	Default: “Untitled Quote”
C	materialsCost	
D	equipmentCost	
E	laborHoursInput	Default: 10
F	calculatedPrice	
G	calculatedGrossMargin	
H	calculatedEstimatedLabor	
I	calculatedMePercent	
J	subcontractorCost	Default: 0
K	craneCost	Default: 0
L	permitCost	Default: 0
M	createdAt	Timestamp string

Sheet: `Locations`

Tax region reference library.

Column	Header	Notes
A	id	Auto-increment
B	name	e.g., “Gaithersburg, MD”
C	salesTaxPercent	e.g., 6.00
D	createdAt	Timestamp

Seed data: | Name | Tax | |——|—–| | Gaithersburg, MD | 6.00 | | Hartford, CT | 6.35 | | Gypsum, CO | 7.40 | | Jupiter, FL | 7.00 | | Denver, CO | 9.15 | | Houston, TX | 8.25 |

Sheet: `Tenants`

ServiceTitan API credentials.

Column	Header	Notes
A	id	Auto-increment
B	name	Display name
C	tenantId	ServiceTitan tenant ID

Column	Header	Notes
D	clientId	OAuth client ID
E	clientSecret	OAuth client secret
F	appKey	ServiceTitan app key
G	locationId	FK to Locations sheet (id)
H	isActive	TRUE/FALSE
I	sortOrder	Integer for display ordering
J	createdAt	Timestamp
K	updatedAt	Timestamp

Sheet: `PushHistory`

Audit log for ServiceTitan push operations.

Column	Header	Notes
A	id	Auto-increment
B	tenantId	FK to Tenants
C	status	pending / in_progress / completed / failed / cancelled / reverted
D	categoryIds	JSON string array
E	targetMePercent	Snapshot of setting used
F	salesTaxPercent	
G	laborMode	
H	laborHourlyRate	
I	laborHours	Default: 8
J	laborPercent	
K	laborBurdenPercent	
L	commissionPercent	
M	commissionBurdenPercent	
N	financingFeePercent	
O	roundUp	TRUE/FALSE
P	processedCount	Default: 0
Q	updatedCount	Default: 0
R	skippedCount	Default: 0
S	errorCount	Default: 0
T	errorMessages	JSON string array
U	originalValues	JSON: [{serviceId, originalPrice, originalCost}]
V	createdAt	
W	startedAt	
X	completedAt	

3. Apps Script Project Structure

```
PricebookCalc (Apps Script Project)
├── Code.gs                    # Main entry point (doGet, routing)
├── Settings.gs                # Settings CRUD
├── Quotes.gs                  # Quote CRUD
├── Locations.gs               # Tax region CRUD
├── Tenants.gs                 # Tenant CRUD
├── PushHistory.gs             # Push history CRUD
├── ServiceTitan.gs            # ServiceTitan API integration
├── PushProcessor.gs           # Push execution logic
├── Auth.gs                    # Password/PIN verification
├── SheetHelpers.gs            # Shared spreadsheet utilities
├── SystemCalculator.html      # System Calculator page
├── RepairCalculator.html      # Repair Calculator page
├── ReverseCalculator.html     # Reverse Calculator page
├── PushToServiceTitan.html    # Push page with tabs
├── PushHistory.html           # Push history page
├── History.html               # Saved quotes page
├── Settings.html              # Settings page
├── Tenants.html               # Tenant management page
├── Locations.html             # Location management page
├── Navigation.html            # Shared sidebar/nav partial
├── Styles.html                # Shared CSS (included via template)
└── ClientUtils.html           # Shared client-side JS utilities
```

---

4. System Calculator

Inputs

Input	Type	Notes
Target M&E %	Dropdown	10%–50% in 0.5% steps
Materials Cost	Currency	Dollar amount
Equipment Cost	Currency	Dollar amount
Labor Hours or Labor %	Number	Depends on laborMode setting
Subcontractor	Currency	Added to price
Crane	Currency	Added to price
Permit	Currency	Added to price
Discount	Currency or %	Toggle between dollar and percent mode
OEM Rebate	Currency	Added to revenue for GM%, not price
Utility Rebate	Currency	Added to revenue for GM%, not price

Calculation Logic (implement in client-side JavaScript)

```
function calculateSystemPrice(inputs, settings) {
  var materials = parseFloat(inputs.materials) || 0;
  var equipment = parseFloat(inputs.equipment) || 0;
  var salesTaxRate = parseFloat(settings.salesTaxPercent) / 100;
  var targetMeRate = parseFloat(inputs.targetMe) / 100;

  // Step 1: Tax & M&E
  var tax = (materials + equipment) * salesTaxRate;
  var totalMe = materials + equipment + tax;

  // Step 2: Base Price from M&E Target
  var basePrice = targetMeRate > 0 ? totalMe / targetMeRate : 0;

  // Step 3: Add Other COGS, Apply Discount
  var subcontractor = parseFloat(inputs.subcontractor) || 0;
  var crane = parseFloat(inputs.crane) || 0;
  var permit = parseFloat(inputs.permit) || 0;
  var totalOtherCogs = subcontractor + crane + permit;
  var priceBeforeDiscount = basePrice + totalOtherCogs;

  var discountVal;
  if (inputs.discountMode === "percent") {
    discountVal = priceBeforeDiscount * (parseFloat(inputs.discount) || 0) / 100;
  } else {
    discountVal = parseFloat(inputs.discount) || 0;
  }
  var price = Math.max(0, priceBeforeDiscount - discountVal);

  // Step 4: Commission (always % of job, with burden)
  var commissionRate = parseFloat(settings.commissionPercent) / 100;
  var commissionBurdenRate = parseFloat(settings.commissionBurdenPercent) / 100;
  var commissionCost = (price * commissionRate) * (1 + commissionBurdenRate);

  // Step 5: Financing
  var financingRate = parseFloat(settings.financingFeePercent) / 100;
  var financingCost = price * financingRate;

  // Step 6: Labor Cost
  var laborBurdenRate = parseFloat(settings.laborBurdenPercent) / 100;
  var laborCost;
  if (settings.laborMode === "hours") {
    var hourlyRate = parseFloat(settings.laborHourlyRate);
    var hours = parseFloat(inputs.laborHours) || 0;
    laborCost = (hours * hourlyRate) * (1 + laborBurdenRate);
  } else {
    var laborPercentRate = parseFloat(inputs.laborPercent) / 100;
```

```
    laborCost = (price * laborPercentRate) * (1 + laborBurdenRate);
  }

  // Step 7: Estimated Labor (for ServiceTitan)
  var estimatedLabor = laborCost + commissionCost + tax + financingCost;

  // Step 8: Display Price & Gross Margin
  var displayPrice = inputs.roundUp ? Math.ceil(price) : price;
  var oemRebate = parseFloat(inputs.oemRebate) || 0;
  var utilityRebate = parseFloat(inputs.utilityRebate) || 0;
  var totalAllCogs = totalMe + commissionCost + financingCost + laborCost + totalO
  var effectiveRevenue = displayPrice + oemRebate + utilityRebate;
  var grossMarginAmount = effectiveRevenue - totalAllCogs;
  var grossMarginPercent = effectiveRevenue > 0
    ? (grossMarginAmount / effectiveRevenue) * 100 : 0;
  var mePercent = displayPrice > 0 ? (totalMe / displayPrice) * 100 : 0;

  return {
    price: displayPrice,
    estimatedLabor: estimatedLabor,
    grossMarginPercent: grossMarginPercent,
    grossMarginAmount: grossMarginAmount,
    mePercent: mePercent,
    breakdown: {
      materials: materials,
      equipment: equipment,
      tax: tax,
      totalMe: totalMe,
      laborCost: laborCost,
      commissionCost: commissionCost,
      financingCost: financingCost,
      subcontractor: subcontractor,
      crane: crane,
      permit: permit,
      discount: discountVal,
      oemRebate: oemRebate,
      utilityRebate: utilityRebate,
      totalAllCogs: totalAllCogs
    }
  };
}
```

Margin Health Indicator

```
function getMarginHealth(gmPercent, grossMarginMinPercent) {
  if (gmPercent >= grossMarginMinPercent) return { label: "Strong", color: "green"
  if (gmPercent >= grossMarginMinPercent - 2) return { label: "Caution", color: "a
  return { label: "Low", color: "red" };
```

```
}
```

---

5. Repair Calculator

Inputs

Input	Type	Notes
Parts Cost	Currency	Raw cost of parts
Material Markup	Dropdown	10%–300%
Labor Rate	Currency	Dollar per hour
Labor Hours	Number	Number of hours
Burden %	Percentage	Employer burden on labor/commission
Sales Tax %	Percentage	Tax on parts
Commission %	Percentage	% of total selling price
Financing Fee %	Percentage	% of total selling price
Highest Tech Pay Rate	Currency	For COGS calculation
Production Pay toggle	Switch	“$ + Hours” vs “% of Job”
PSI Mode toggle	Switch	Preset PSI values

Calculation Logic

```
function calculateRepairPrice(inputs) {
  var partsCost = parseFloat(inputs.partsCost) || 0;
  var markupRate = parseFloat(inputs.materialMarkup) / 100;
  var laborRate = parseFloat(inputs.laborRate) || 0;
  var laborHours = parseFloat(inputs.laborHours) || 0;
  var burdenRate = parseFloat(inputs.burdenPercent) / 100;
  var taxRate = parseFloat(inputs.salesTaxPercent) / 100;
  var commissionRate = parseFloat(inputs.commissionPercent) / 100;
  var financingRate = parseFloat(inputs.financingFeePercent) / 100;
  var highestTechRate = parseFloat(inputs.highestTechRate) || 0;
  var roundUp = inputs.roundUp;
  var laborMode = inputs.laborMode; // "dollars" or "percent"

  // Parts + Tax + Markup
  var salesTax = partsCost * taxRate;
  var subtotal1 = partsCost + salesTax;
  var markupAmount = subtotal1 * markupRate;
  var subtotal2 = subtotal1 + markupAmount;

  // Labor
  var directLaborCost, burdenedLaborCost;
  if (laborMode === "percent") {
    directLaborCost = 0;
    burdenedLaborCost = 0;
```

```
  } else {
    directLaborCost = laborRate * laborHours;
    var laborBurden = directLaborCost * burdenRate;
    burdenedLaborCost = directLaborCost + laborBurden;
  }
  var subtotal3 = subtotal2 + burdenedLaborCost;

  // Solve for total price (commission and financing are % of total)
  var denominator = 1 - commissionRate * (1 + burdenRate) - financingRate;
  var totalPriceCalc = denominator > 0 ? subtotal3 / denominator : subtotal3;

  // Commission & Financing
  var directCommission = totalPriceCalc * commissionRate;
  var commissionBurden = directCommission * burdenRate;
  var burdenedCommission = directCommission + commissionBurden;
  var financingFee = totalPriceCalc * financingRate;

  // Total
  var totalBeforeRounding = subtotal3 + burdenedCommission + financingFee;
  var totalPrice = roundUp ? Math.ceil(totalBeforeRounding) : totalBeforeRounding;

  // COGS & Gross Margin
  var techLaborCost = laborMode === "percent" ? 0 : highestTechRate * laborHours;
  var burdenedTechLabor = techLaborCost * (1 + burdenRate);
  var cogs = partsCost + burdenedTechLabor + burdenedCommission;
  var grossMargin = totalPrice - cogs;
  var grossMarginPercent = totalPrice > 0 ? (grossMargin / totalPrice) * 100 : 0;

  return {
    totalPrice: totalPrice,
    grossMarginPercent: grossMarginPercent,
    grossMarginAmount: grossMargin,
    breakdown: {
      partsCost: partsCost,
      salesTax: salesTax,
      markupAmount: markupAmount,
      directLaborCost: directLaborCost,
      burdenedLaborCost: burdenedLaborCost,
      directCommission: directCommission,
      burdenedCommission: burdenedCommission,
      financingFee: financingFee,
      cogs: cogs
    }
  };
}
```

Margin Health (Repair uses fixed thresholds)

```
function getRepairMarginHealth(gmPercent) {
  if (gmPercent >= 50) return { label: "Strong", color: "green" };
  if (gmPercent >= 40) return { label: "Caution", color: "amber" };
  return { label: "Low", color: "red" };
}
```

---

6. Reverse Calculator

Works backwards from a known selling price to determine the resulting gross margin. Has two modes: System and Repair.

System Mode

Same inputs as System Calculator, but replaces Target M&E % with a Selling Price input. Outputs GM% instead of computing a price.

```
function calculateReverseSystem(inputs, settings) {
  var sellingPrice = parseFloat(inputs.sellingPrice) || 0;
  var materials = parseFloat(inputs.materials) || 0;
  var equipment = parseFloat(inputs.equipment) || 0;
  var salesTaxRate = parseFloat(settings.salesTaxPercent) / 100;

  var tax = (materials + equipment) * salesTaxRate;
  var totalMe = materials + equipment + tax;

  var subcontractor = parseFloat(inputs.subcontractor) || 0;
  var crane = parseFloat(inputs.crane) || 0;
  var permit = parseFloat(inputs.permit) || 0;
  var totalOtherCogs = subcontractor + crane + permit;

  // Discount
  var discountVal;
  if (inputs.discountMode === "percent") {
    discountVal = sellingPrice * (parseFloat(inputs.discount) || 0) / 100;
  } else {
    discountVal = parseFloat(inputs.discount) || 0;
  }
  var price = Math.max(0, sellingPrice - discountVal);

  // Commission, Financing, Labor (same formulas as System Calculator)
  var commissionRate = parseFloat(settings.commissionPercent) / 100;
  var commissionBurdenRate = parseFloat(settings.commissionBurdenPercent) / 100;
  var commissionCost = (price * commissionRate) * (1 + commissionBurdenRate);
  var financingRate = parseFloat(settings.financingFeePercent) / 100;
  var financingCost = price * financingRate;
```

```
  var laborBurdenRate = parseFloat(settings.laborBurdenPercent) / 100;
  var laborCost;
  if (settings.laborMode === "hours") {
    laborCost = (parseFloat(inputs.laborHours) || 0)
      * parseFloat(settings.laborHourlyRate) * (1 + laborBurdenRate);
  } else {
    laborCost = (price * (parseFloat(inputs.laborPercent) / 100)) * (1 + laborBurd
  }

  var oemRebate = parseFloat(inputs.oemRebate) || 0;
  var utilityRebate = parseFloat(inputs.utilityRebate) || 0;
  var totalAllCogs = totalMe + commissionCost + financingCost + laborCost + totalO
  var effectiveRevenue = price + oemRebate + utilityRebate;
  var grossMarginAmount = effectiveRevenue - totalAllCogs;
  var grossMarginPercent = effectiveRevenue > 0
    ? (grossMarginAmount / effectiveRevenue) * 100 : 0;
  var mePercent = effectiveRevenue > 0 ? (totalMe / effectiveRevenue) * 100 : 0;

  return {
    grossMarginPercent: grossMarginPercent,
    grossMarginAmount: grossMarginAmount,
    mePercent: mePercent,
    totalAllCogs: totalAllCogs,
    effectiveRevenue: effectiveRevenue
  };
}
```

Repair Mode

Same as Repair Calculator inputs but uses a user-entered selling price instead of computing one.

```
function calculateReverseRepair(inputs) {
  var sellingPrice = parseFloat(inputs.sellingPrice) || 0;
  var partsCost = parseFloat(inputs.partsCost) || 0;
  var taxRate = parseFloat(inputs.salesTaxPercent) / 100;
  var markupRate = parseFloat(inputs.materialMarkup) / 100;
  var burdenRate = parseFloat(inputs.burdenPercent) / 100;
  var commissionRate = parseFloat(inputs.commissionPercent) / 100;
  var financingRate = parseFloat(inputs.financingFeePercent) / 100;
  var highestTechRate = parseFloat(inputs.highestTechRate) || 0;
  var laborHours = parseFloat(inputs.laborHours) || 0;

  var repairSalesTax = partsCost * taxRate;
  var repairMaterialsCost = (partsCost + repairSalesTax) * (1 + markupRate);

  var techLaborCost = inputs.laborMode === "percent" ? 0 : highestTechRate * labor
```

```
  var burdenedTechLabor = techLaborCost * (1 + burdenRate);

  var directCommission = sellingPrice * commissionRate;
  var burdenedCommission = directCommission * (1 + burdenRate);

  var cogs = partsCost + burdenedTechLabor + burdenedCommission;
  var grossMargin = sellingPrice - cogs;
  var grossMarginPercent = sellingPrice > 0 ? (grossMargin / sellingPrice) * 100 :

  return {
    grossMarginPercent: grossMarginPercent,
    grossMarginAmount: grossMargin,
    cogs: cogs
  };
}
```

---

7. Settings Management

Server-Side (Settings.gs)

```
function getSettings() {
  var ss = SpreadsheetApp.openById(SPREADSHEET_ID);
  var sheet = ss.getSheetByName("Settings");
  var data = sheet.getDataRange().getValues();

  if (data.length < 2) {
    // Create default row
    var defaults = [1, 23.00, 6.35, "percent", 0.00, 7.00, 20.00,
                    10.00, 20.00, 6.00, 52.00, "ZephyrHome160", "7437"];
    sheet.appendRow(defaults);
    data = sheet.getDataRange().getValues();
  }

  var headers = data[0];
  var values = data[1];
  var settings = {};
  headers.forEach(function(h, i) { settings[h] = values[i]; });
  return settings;
}

function updateSettings(newSettings) {
  var ss = SpreadsheetApp.openById(SPREADSHEET_ID);
  var sheet = ss.getSheetByName("Settings");
  var headers = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0];

  headers.forEach(function(h, i) {
```

```
    if (newSettings.hasOwnProperty(h)) {
      sheet.getRange(2, i + 1).setValue(newSettings[h]);
    }
  });

  return getSettings();
}
```

Per-Device Cache (Client-Side)

Same approach as the original app — use `localStorage` to cache settings per browser/device.

```
var LOCAL_STORAGE_KEY = "pricebookcalc_settings";

function loadLocalSettings() {
  var stored = localStorage.getItem(LOCAL_STORAGE_KEY);
  return stored ? JSON.parse(stored) : null;
}

function saveLocalSettings(settings) {
  localStorage.setItem(LOCAL_STORAGE_KEY, JSON.stringify(settings));
}
```

---

8. Quote Saving & History

Server-Side (Quotes.gs)

```
function getQuotes() {
  var sheet = getSheet("Quotes");
  return sheetToArray(sheet); // Helper that returns array of row objects
}

function createQuote(data) {
  var sheet = getSheet("Quotes");
  var id = getNextId(sheet);
  sheet.appendRow([
    id,
    data.title || "Untitled Quote",
    data.materialsCost,
    data.equipmentCost,
    data.laborHoursInput || 10,
    data.calculatedPrice,
    data.calculatedGrossMargin,
    data.calculatedEstimatedLabor,
    data.calculatedMePercent,
    data.subcontractorCost || 0,
```

```
    data.craneCost || 0,
    data.permitCost || 0,
    new Date().toISOString()
  ]);
  return { id: id };
}

function deleteQuote(id) {
  deleteRowById(getSheet("Quotes"), id);
}

function clearAllQuotes() {
  var sheet = getSheet("Quotes");
  if (sheet.getLastRow() > 1) {
    sheet.deleteRows(2, sheet.getLastRow() - 1);
  }
}
```

---

9. Authentication & Access Control

The original app uses IP-based session tracking. In Apps Script, use a different approach since you have Google account authentication built-in.

Option A: Google Account Authentication (Recommended)

Deploy the web app as "Execute as: Me" and "Who has access: Anyone with Google account." Users must be signed into Google. Use allowlists for admin access.

```
function checkAccess() {
  var email = Session.getActiveUser().getEmail();
  var admins = PropertiesService.getScriptProperties().getProperty("ADMIN_EMAILS")
  return {
    email: email,
    isAdmin: admins && admins.indexOf(email) >= 0
  };
}
```

Option B: Password/PIN Matching (Original Behavior)

If you want to replicate the original password/PIN gates without Google account restrictions, use session-based tokens stored in CacheService.

```
function verifyPassword(password) {
  var settings = getSettings();
  if (password === settings.serviceTitanPassword) {
    var sessionId = Utilities.getUuid();
    CacheService.getScriptCache().put("session_" + sessionId, "true", 28800); // 8
```

```
    return { success: true, sessionId: sessionId };
  }
  return { success: false };
}

function checkSession(sessionId) {
  var cached = CacheService.getScriptCache().get("session_" + sessionId);
  return { unlocked: cached === "true" };
}

function verifyPin(pin) {
  var settings = getSettings();
  if (pin === settings.adminPin) {
    var sessionId = Utilities.getUuid();
    CacheService.getScriptCache().put("admin_" + sessionId, "true", 28800);
    return { success: true, sessionId: sessionId };
  }
  return { success: false };
}
```

Client-Side Session Storage

```
function storeSession(key, sessionId) {
  sessionStorage.setItem(key, sessionId);
}

function getSession(key) {
  return sessionStorage.getItem(key);
}
```

---

10. ServiceTitan Integration

OAuth Token Management (ServiceTitan.gs)

```
var ST_AUTH_URL = "https://auth.servicetitan.io/connect/token";
var ST_API_BASE = "https://api.servicetitan.io";

function getServiceTitanToken(tenant) {
  var cache = CacheService.getScriptCache();
  var cacheKey = "st_token_" + tenant.tenantId;
  var cachedToken = cache.get(cacheKey);
  if (cachedToken) return cachedToken;

  var response = UrlFetchApp.fetch(ST_AUTH_URL, {
    method: "post",
    contentType: "application/x-www-form-urlencoded",
```

```
    payload: {
      grant_type: "client_credentials",
      client_id: tenant.clientId,
      client_secret: tenant.clientSecret
    },
    muteHttpExceptions: true
  });

  var result = JSON.parse(response.getContentText());
  if (result.access_token) {
    cache.put(cacheKey, result.access_token, 900); // 15 minutes
    return result.access_token;
  }
  throw new Error("Failed to get ServiceTitan token: " + response.getContentText()
}
```

API Request Helper

```
function stApiRequest(method, path, tenant, payload) {
  var token = getServiceTitanToken(tenant);
  var options = {
    method: method,
    headers: {
      "Authorization": "Bearer " + token,
      "ST-App-Key": tenant.appKey,
      "Content-Type": "application/json"
    },
    muteHttpExceptions: true
  };
  if (payload) options.payload = JSON.stringify(payload);

  var url = ST_API_BASE + path;
  var response = UrlFetchApp.fetch(url, options);
  var code = response.getResponseCode();

  if (code === 429) {
    // Rate limited - wait and retry
    Utilities.sleep(2000);
    return stApiRequest(method, path, tenant, payload);
  }

  return JSON.parse(response.getContentText());
}
```

Key API Methods

```
function getCategories(tenant) {
  return stApiRequest("get",
```

```
    "/pricebook/v2/tenant/" + tenant.tenantId + "/categories?pageSize=500",
    tenant);
}

function getServices(tenant, categoryId) {
  return stApiRequest("get",
    "/pricebook/v2/tenant/" + tenant.tenantId + "/services?categoryId="
    + categoryId + "&pageSize=500", tenant);
}

function getMaterials(tenant, page, pageSize) {
  return stApiRequest("get",
    "/pricebook/v2/tenant/" + tenant.tenantId + "/materials?page="
    + page + "&pageSize=" + (pageSize || 500), tenant);
}

function getEquipment(tenant, page, pageSize) {
  return stApiRequest("get",
    "/pricebook/v2/tenant/" + tenant.tenantId + "/equipment?page="
    + page + "&pageSize=" + (pageSize || 500), tenant);
}

function updateService(tenant, serviceId, data) {
  return stApiRequest("patch",
    "/pricebook/v2/tenant/" + tenant.tenantId + "/services/" + serviceId,
    tenant, data);
}

function createService(tenant, data) {
  return stApiRequest("post",
    "/pricebook/v2/tenant/" + tenant.tenantId + "/services",
    tenant, data);
}

function getGLAccounts(tenant) {
  return stApiRequest("get",
    "/accounting/v2/tenant/" + tenant.tenantId + "/gl-accounts?pageSize=500",
    tenant);
}
```

---

11. Push to ServiceTitan

Push Processor Logic (PushProcessor.gs)

The push operation calculates new prices for ServiceTitan services based on their linked materials and equipment costs.

```
function executePush(tenantId, categoryIds, pushSettings) {
  var tenant = getTenantById(tenantId);
  var pushId = createPushHistoryRecord(tenantId, categoryIds, pushSettings);

  updatePushStatus(pushId, "in_progress", { startedAt: new Date().toISOString() })

  try {
    // 1. Fetch all materials and equipment for cost lookup
    var allMaterials = fetchAllPaginated(function(page) {
      return getMaterials(tenant, page, 500);
    });
    var allEquipment = fetchAllPaginated(function(page) {
      return getEquipment(tenant, page, 500);
    });

    // Build lookup maps
    var materialMap = {};
    allMaterials.forEach(function(m) { materialMap[m.id] = m; });
    var equipmentMap = {};
    allEquipment.forEach(function(e) { equipmentMap[e.id] = e; });

    // 2. Fetch services from selected categories
    var services = [];
    categoryIds.forEach(function(catId) {
      var result = getServices(tenant, catId);
      if (result.data) services = services.concat(result.data);
    });

    // 3. Process each service
    var processed = 0, updated = 0, skipped = 0, errors = 0;
    var originalValues = [];
    var salesTaxRate = parseFloat(pushSettings.salesTaxPercent) / 100;
    var targetMeRate = parseFloat(pushSettings.targetMePercent) / 100;

    services.forEach(function(service) {
      processed++;
      try {
        // Calculate M&E cost from linked materials and equipment
        var materialsCost = 0;
        if (service.serviceMaterials) {
          service.serviceMaterials.forEach(function(sm) {
            var mat = materialMap[sm.skuId];
            if (mat) materialsCost += mat.cost * (sm.quantity || 1);
          });
        }
        var equipmentCost = 0;
        if (service.serviceEquipment) {
          service.serviceEquipment.forEach(function(se) {
```

```
            var eq = equipmentMap[se.skuId];
            if (eq) equipmentCost += eq.cost * (se.quantity || 1);
          });
        }

        var rawMeCost = materialsCost + equipmentCost;
        if (rawMeCost <= 0) { skipped++; return; }

        var tax = rawMeCost * salesTaxRate;
        var totalMe = rawMeCost + tax;
        var newPrice = targetMeRate > 0 ? totalMe / targetMeRate : 0;

        if (pushSettings.roundUp) newPrice = Math.ceil(newPrice);

        // Save original for revert
        originalValues.push({
          serviceId: service.id,
          originalPrice: service.price,
          originalCost: service.cost
        });

        // Update via API
        updateService(tenant, service.id, { price: newPrice });
        updated++;
      } catch (e) {
        errors++;
      }
    });

    updatePushStatus(pushId, "completed", {
      processedCount: processed,
      updatedCount: updated,
      skippedCount: skipped,
      errorCount: errors,
      originalValues: JSON.stringify(originalValues),
      completedAt: new Date().toISOString()
    });

  } catch (e) {
    updatePushStatus(pushId, "failed", {
      errorMessages: JSON.stringify([e.message]),
      completedAt: new Date().toISOString()
    });
  }

  return pushId;
}
```

Rate Limiting

Apps Script has a 6-minute execution limit. For large pushes: - Process in batches and use `Utilities.sleep(100)` between API calls - If needed, use time-driven triggers to continue processing across multiple executions - Store progress in the PushHistory sheet and resume from where you left off

---

12. Tax Regions (Locations)

Server-Side (Locations.gs)

```
function getLocations() {
  var sheet = getSheet("Locations");
  var data = sheetToArray(sheet);
  data.sort(function(a, b) { return a.name.localeCompare(b.name); });
  return data;
}

function createLocation(data) {
  var sheet = getSheet("Locations");
  var id = getNextId(sheet);
  sheet.appendRow([id, data.name, data.salesTaxPercent, new Date().toISOString()])
  return { id: id };
}

function updateLocation(id, data) {
  var sheet = getSheet("Locations");
  var rowIndex = findRowById(sheet, id);
  if (rowIndex < 0) throw new Error("Location not found");
  if (data.name !== undefined) sheet.getRange(rowIndex, 2).setValue(data.name);
  if (data.salesTaxPercent !== undefined) sheet.getRange(rowIndex, 3).setValue(dat
}

function deleteLocation(id) {
  // Clear locationId on any tenants using this location
  var tenantSheet = getSheet("Tenants");
  var tenants = sheetToArray(tenantSheet);
  tenants.forEach(function(t, i) {
    if (t.locationId == id) {
      var rowIndex = i + 2; // row 1 is headers
      tenantSheet.getRange(rowIndex, 7).setValue(""); // Column G = locationId
    }
  });
  deleteRowById(getSheet("Locations"), id);
}
```

Tax Auto-Population on Tenant Selection

When a tenant is selected on the Push page, look up the tenant's assigned locationId, find the matching location, and populate the sales tax field in all calculator sections.

```
// Client-side
function onTenantSelected(tenantId) {
  google.script.run.withSuccessHandler(function(result) {
    if (result.salesTaxPercent) {
      // Update all three tab tax fields
      document.getElementById("updateTaxRate").value = result.salesTaxPercent;
      document.getElementById("systemTaxRate").value = result.salesTaxPercent;
      document.getElementById("repairTaxRate").value = result.salesTaxPercent;
    }
  }).getTenantTaxRate(tenantId);
}

// Server-side
function getTenantTaxRate(tenantId) {
  var tenant = getTenantById(tenantId);
  if (tenant && tenant.locationId) {
    var locations = getLocations();
    var loc = locations.find(function(l) { return l.id == tenant.locationId; });
    if (loc) return { salesTaxPercent: loc.salesTaxPercent };
  }
  var settings = getSettings();
  return { salesTaxPercent: settings.salesTaxPercent || "6.35" };
}
```

---

13. Tenant Management

Server-Side (Tenants.gs)

```
function getTenants() {
  var tenants = sheetToArray(getSheet("Tenants"));
  // Mask credentials for display
  return tenants.map(function(t) {
    return {
      id: t.id,
      name: t.name,
      tenantId: t.tenantId,
      clientId: maskString(t.clientId),
      clientSecret: maskString(t.clientSecret),
      appKey: maskString(t.appKey),
      locationId: t.locationId,
      isActive: t.isActive,
      sortOrder: t.sortOrder
```

```
    };
  });
}

function getTenantFull(id) {
  // Returns unmasked credentials (use behind auth gate)
  return sheetToArray(getSheet("Tenants")).find(function(t) { return t.id == id; });
}

function getTenantById(id) {
  return sheetToArray(getSheet("Tenants")).find(function(t) { return t.id == id; });
}

function createTenant(data) {
  var sheet = getSheet("Tenants");
  var id = getNextId(sheet);
  var now = new Date().toISOString();
  sheet.appendRow([
    id, data.name, data.tenantId, data.clientId, data.clientSecret,
    data.appKey, data.locationId || "", true, data.sortOrder || 0, now, now
  ]);
  return { id: id };
}

function updateTenant(id, data) {
  var sheet = getSheet("Tenants");
  var rowIndex = findRowById(sheet, id);
  if (rowIndex < 0) throw new Error("Tenant not found");
  var headers = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0];
  headers.forEach(function(h, i) {
    if (data.hasOwnProperty(h) && h !== "id") {
      sheet.getRange(rowIndex, i + 1).setValue(data[h]);
    }
  });
  sheet.getRange(rowIndex, headers.indexOf("updatedAt") + 1).setValue(new Date().t
}

function deleteTenant(id) {
  deleteRowById(getSheet("Tenants"), id);
}

function maskString(str) {
  if (!str || str.length < 8) return "****";
  return str.substring(0, 4) + "****" + str.substring(str.length - 4);
}
```

Connection Test

```
function testTenantConnection(tenantId) {
  try {
    var tenant = getTenantById(tenantId);
    var result = getCategories(tenant);
    return { success: true, categoryCount: result.data ? result.data.length : 0 };
  } catch (e) {
    return { success: false, error: e.message };
  }
}
```

---

14. Push History & Revert

Server-Side (PushHistory.gs)

```
function getPushHistory(tenantId) {
  var all = sheetToArray(getSheet("PushHistory"));
  if (tenantId) {
    all = all.filter(function(r) { return r.tenantId == tenantId; });
  }
  all.sort(function(a, b) { return new Date(b.createdAt) - new Date(a.createdAt); });
  return all;
}

function createPushHistoryRecord(tenantId, categoryIds, settings) {
  var sheet = getSheet("PushHistory");
  var id = getNextId(sheet);
  sheet.appendRow([
    id, tenantId, "pending", JSON.stringify(categoryIds),
    settings.targetMePercent, settings.salesTaxPercent,
    settings.laborMode, settings.laborHourlyRate,
    settings.laborHours || 8, settings.laborPercent,
    settings.laborBurdenPercent, settings.commissionPercent,
    settings.commissionBurdenPercent, settings.financingFeePercent,
    settings.roundUp || false,
    0, 0, 0, 0, "", "", new Date().toISOString(), "", ""
  ]);
  return id;
}

function updatePushStatus(id, status, updates) {
  var sheet = getSheet("PushHistory");
  var rowIndex = findRowById(sheet, id);
  if (rowIndex < 0) return;
  var headers = sheet.getRange(1, 1, 1, sheet.getLastColumn()).getValues()[0];
```

```
  sheet.getRange(rowIndex, headers.indexOf("status") + 1).setValue(status);
  if (updates) {
    Object.keys(updates).forEach(function(key) {
      var colIndex = headers.indexOf(key);
      if (colIndex >= 0) sheet.getRange(rowIndex, colIndex + 1).setValue(updates[k
    });
  }
}

function revertPush(pushId) {
  var record = sheetToArray(getSheet("PushHistory")).find(function(r) { return r.i
  if (!record || !record.originalValues) throw new Error("No revert data available

  var tenant = getTenantById(record.tenantId);
  var originals = JSON.parse(record.originalValues);

  originals.forEach(function(orig) {
    updateService(tenant, orig.serviceId, { price: orig.originalPrice });
  });

  updatePushStatus(pushId, "reverted", { completedAt: new Date().toISOString() });
}
```

---

15. PSI Mode Toggle

PSI Mode applies preset values for PSI (Performance Service Institute) pricing standards.

New System Task PSI Mode

- Switches GL Account from “Revenue” to “Sales”

New Repair Task PSI Mode

Setting	PSI ON	PSI OFF (Default)
Material Markup	40%	200%
Labor Rate	$600	$175
Sales Tax	7.4%	From tenant’s location (fallback 6.35%)
GL Account	“Sales”	“Revenue”
Financing Fee	(unchanged)	4%

Client-Side Implementation

```
function togglePsiMode(enabled, tenantLocationTax) {
  if (enabled) {
```

```
    document.getElementById("materialMarkup").value = "40";
    document.getElementById("laborRate").value = "600";
    document.getElementById("repairTaxRate").value = "7.4";
    setGlAccount("Sales");
  } else {
    document.getElementById("materialMarkup").value = "200";
    document.getElementById("laborRate").value = "175";
    document.getElementById("repairTaxRate").value = tenantLocationTax || "6.35";
    document.getElementById("financingFee").value = "4";
    setGlAccount("Revenue");
  }
  recalculate();
}
```

---

16. Commission & Labor Modes

Commission

Always calculated as "% of Job" — a percentage of the total selling price backed into the total. No toggle.

```
directCommission = totalPrice * commissionRate
commissionBurden = directCommission * burdenRate
burdenedCommission = directCommission + commissionBurden
```

Labor Mode (Production Pay Toggle)

"$ + Hours" Mode (default): - Labor Cost = Labor Rate x Labor Hours - Commission is a separate % of job - Formula: `T = subtotal3 / (1 - commissionRate*(1+burdenRate) - financingRate)`

"% of Job" Mode: - Labor cost = $0 (commission handles everything) - Commission % covers both labor and commission combined - Formula: `T = subtotal2 / (1 - commissionRate*(1+burdenRate) - financingRate)` - Hide Labor line in breakdown, relabel Commission as "Commission incl. Labor" - Highest Tech Pay Rate and COGS tech labor are zeroed

---

17. UI Implementation with HTML Service

Page Routing (Code.gs)

```
var SPREADSHEET_ID = "YOUR_SPREADSHEET_ID_HERE";

function doGet(e) {
  var page = (e && e.parameter && e.parameter.page) ? e.parameter.page : "calculat
```

```
  var pageMap = {
    "calculator": "SystemCalculator",
    "repair": "RepairCalculator",
    "reverse": "ReverseCalculator",
    "push": "PushToServiceTitan",
    "push-history": "PushHistory",
    "history": "History",
    "settings": "Settings",
    "tenants": "Tenants",
    "locations": "Locations"
  };

  var templateName = pageMap[page] || "SystemCalculator";
  var template = HtmlService.createTemplateFromFile(templateName);
  template.currentPage = page;

  return template.evaluate()
    .setTitle("PricebookCalc")
    .setFaviconUrl("YOUR_FAVICON_URL")
    .addMetaTag("viewport", "width=device-width, initial-scale=1.0, maximum-scale=
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}

function include(filename) {
  return HtmlService.createHtmlOutputFromFile(filename).getContent();
}
```

HTML Template Pattern

Each page HTML file follows this pattern:

```
<!DOCTYPE html>
<html>
<head>
  <?!= include("Styles"); ?>
</head>
<body>
  <div class="app-layout">
    <?!= include("Navigation"); ?>
    <main class="main-content">
      <!-- Page-specific content here -->
    </main>
  </div>
  <?!= include("ClientUtils"); ?>
  <script>
    // Page-specific JavaScript
  </script>
</body>
```

```
</html>
```

Navigation Partial (Navigation.html)

```
<nav class="sidebar" id="sidebar">
  <div class="sidebar-header">
    <img src="YOUR_LOGO_URL" alt="PricebookCalc" class="logo">
  </div>
  <div class="nav-links">
    <a href="?page=calculator" class="nav-link <?= currentPage === 'calculator' ?
      System Calculator
    </a>
    <a href="?page=repair" class="nav-link <?= currentPage === 'repair' ? 'active'
      Repair Calculator
    </a>
    <a href="?page=reverse" class="nav-link <?= currentPage === 'reverse' ? 'activ
      Reverse Calculator
    </a>

    <!-- ServiceTitan Section (collapsible, password-gated) -->
    <div class="nav-section">
      <div class="nav-section-header" onclick="toggleSection('st-section')">
        ServiceTitan
      </div>
      <div id="st-section" class="nav-section-body" style="display:none;">
        <a href="?page=push" class="nav-link">Push to ServiceTitan</a>
        <a href="?page=push-history" class="nav-link">Push History</a>
        <a href="?page=tenants" class="nav-link">Tenants</a>
      </div>
    </div>

    <a href="?page=history" class="nav-link">Saved Quotes</a>
    <a href="?page=settings" class="nav-link">Settings</a>
    <a href="?page=locations" class="nav-link">Locations</a>
  </div>
</nav>

<!-- Mobile header -->
<div class="mobile-header" id="mobileHeader">
  <img src="YOUR_LOGO_STACKED_URL" alt="PricebookCalc" class="mobile-logo">
  <button onclick="toggleSidebar()">&#9776;</button>
</div>
```

Client-Server Communication Pattern

```
// Client-side: Call server functions
function loadSettings() {
```

```
  google.script.run
    .withSuccessHandler(function(settings) {
      // Update UI with settings
      populateForm(settings);
    })
    .withFailureHandler(function(error) {
      showToast("Error loading settings: " + error.message, "error");
    })
    .getSettings();
}

function saveQuote(data) {
  google.script.run
    .withSuccessHandler(function(result) {
      showToast("Quote saved!", "success");
    })
    .withFailureHandler(function(error) {
      showToast("Error: " + error.message, "error");
    })
    .createQuote(data);
}
```

Toast Notification System

```
<div id="toast-container"></div>
<script>
function showToast(message, type) {
  var container = document.getElementById("toast-container");
  var toast = document.createElement("div");
  toast.className = "toast toast-" + (type || "info");
  toast.textContent = message;
  container.appendChild(toast);
  setTimeout(function() { toast.remove(); }, 4000);
}
</script>
```

---

18. Step-by-Step Setup Instructions

Phase 1: Create the Google Sheets Database

1. Create a new Google Spreadsheet
2. Rename the first sheet tab to `Settings`
3. Add header row: `id | targetMePercent | salesTaxPercent | laborMode | laborHourlyRate | laborPercent | laborBurdenPercent | commissionPercent | commissionBurdenPercent | financingFeePercent | grossMarginMinPercent | serviceTitanPassword | adminPin`

4. Add default data row: `1 | 23.00 | 6.35 | percent | 0.00 | 7.00 | 20.00 | 10.00 | 20.00 | 6.00 | 52.00 | ZephyrHome160 | 7437`
5. Create additional sheet tabs: `Quotes`, `Locations`, `Tenants`, `PushHistory`
6. Add headers to each tab matching the column definitions in Section 2
7. Seed the `Locations` tab with the 6 default tax regions
8. Note the spreadsheet ID from the URL (the long string between `/d/` and `/edit`)

Phase 2: Create the Apps Script Project

1. Open Google Apps Script (script.google.com) and create a new project
2. Name it "PricebookCalc"
3. Create the `.gs` files listed in Section 3
4. Create the `.html` files listed in Section 3
5. In `Code.gs`, set `SPREADSHEET_ID` to your spreadsheet's ID
6. Store sensitive config in Script Properties:
    - Go to Project Settings > Script Properties
    - Add `SPREADSHEET_ID` with your spreadsheet ID
    - Add `ADMIN_EMAILS` with comma-separated admin email addresses (if using Google auth)

Phase 3: Implement Core Features (Build Order)

Build in this order to minimize dependencies:

1. **SheetHelpers.gs** — Shared utilities (getSheet, sheetToArray, getNextId, findRowById, deleteRowById)
2. **Settings.gs** — Settings CRUD
3. **Code.gs** — Routing and doGet
4. **Styles.html** — Shared CSS
5. **Navigation.html** — Sidebar navigation
6. **ClientUtils.html** — Shared client utilities (toast, formatting, localStorage)
7. **SystemCalculator.html** — System Calculator (largest page, core feature)
8. **RepairCalculator.html** — Repair Calculator
9. **ReverseCalculator.html** — Reverse Calculator
10. **Settings.html** — Settings page
11. **Quotes.gs + History.html** — Quote saving and history
12. **Auth.gs** — Authentication
13. **Locations.gs + Locations.html** — Tax region management
14. **Tenants.gs + Tenants.html** — Tenant management
15. **ServiceTitan.gs** — ServiceTitan API integration
16. **PushProcessor.gs** — Push execution logic
17. **PushToServiceTitan.html** — Push page with tabs
18. **PushHistory.gs + PushHistory.html** — Push history and revert

Phase 4: Deploy

1. In Apps Script, click Deploy > New deployment
2. Select type: Web app

3. Execute as: Me (your account)
4. Who has access: Anyone with Google account (or Anyone, depending on needs)
5. Click Deploy
6. Copy the web app URL — this is your app's public URL
7. Share the URL with your team

Phase 5: Upload Assets

1. Upload logo images to Google Drive
2. Make them publicly viewable (Anyone with the link)
3. Use the Drive file's direct link format: `https://drive.google.com/uc?id=FILE_ID`
4. Update logo URLs in Navigation.html

---

19. Limitations & Workarounds

Apps Script Execution Limits

Limit	Value	Workaround
Execution time	6 minutes per call	Break large pushes into batches; use time-driven triggers for continuation
Daily URL fetch	20,000 calls	Monitor ServiceTitan API call volume; batch where possible
Cache size	100KB per item	Store large datasets in Sheets instead of cache
HTML Service size	500KB per page	Keep pages modular; lazy-load content
Concurrent users	~30 simultaneous	Acceptable for small team use

No Real-Time Progress Updates

The original app uses Server-Sent Events for live push progress. In Apps Script: - Show a progress modal on the client side - Poll the PushHistory sheet every 3–5 seconds via `google.script.run` to check status - Update the progress UI based on processed-Count/total

```
function pollPushProgress(pushId) {
  google.script.run
    .withSuccessHandler(function(record) {
      updateProgressUI(record);
      if (record.status === "in_progress" || record.status === "pending") {
        setTimeout(function() { pollPushProgress(pushId); }, 3000);
      }
    })
```

```
    .getPushHistoryById(pushId);
}
```

No Pie Charts (Recharts Equivalent)

Use Google Charts (loaded via CDN) for cost breakdown visualization:

```
<script src="https://www.gstatic.com/charts/loader.js"></script>
<script>
google.charts.load('current', {'packages':['corechart']});
function drawPieChart(breakdown) {
  var data = google.visualization.arrayToDataTable([
    ['Category', 'Amount'],
    ['Materials & Equipment', breakdown.totalMe],
    ['Labor', breakdown.laborCost],
    ['Commission', breakdown.commissionCost],
    ['Financing', breakdown.financingCost],
    ['Other COGS', breakdown.subcontractor + breakdown.crane + breakdown.permit]
  ]);
  var chart = new google.visualization.PieChart(document.getElementById('chart'));
  chart.draw(data, { title: 'Cost Distribution', pieHole: 0.3 });
}
</script>
<div id="chart" style="width:100%; height:300px;"></div>
```

No Image Export

The original uses `html-to-image` to export results as PNG. In Apps Script: - Option A: Use `window.print()` with a print-specific CSS stylesheet - Option B: Generate a PDF via Apps Script and serve it as a download - Option C: Include a “Copy to Clipboard” button for key values

Mobile Responsiveness

HTML Service supports responsive design. Use CSS media queries the same way:

```
@media (max-width: 768px) {
  .sidebar { display: none; }
  .mobile-header { display: flex; }
  .main-content { padding: 1rem; }
}
@media (min-width: 769px) {
  .sidebar { display: flex; }
  .mobile-header { display: none; }
}
```

---

20. Appendix: Code Patterns

SheetHelpers.gs — Shared Utilities

```
function getSheet(name) {
  var id = PropertiesService.getScriptProperties().getProperty("SPREADSHEET_ID") |
  return SpreadsheetApp.openById(id).getSheetByName(name);
}

function sheetToArray(sheet) {
  var data = sheet.getDataRange().getValues();
  if (data.length < 2) return [];
  var headers = data[0];
  var results = [];
  for (var i = 1; i < data.length; i++) {
    var row = {};
    headers.forEach(function(h, j) { row[h] = data[i][j]; });
    results.push(row);
  }
  return results;
}

function getNextId(sheet) {
  var data = sheet.getDataRange().getValues();
  if (data.length < 2) return 1;
  var maxId = 0;
  for (var i = 1; i < data.length; i++) {
    var id = parseInt(data[i][0]) || 0;
    if (id > maxId) maxId = id;
  }
  return maxId + 1;
}

function findRowById(sheet, id) {
  var data = sheet.getDataRange().getValues();
  for (var i = 1; i < data.length; i++) {
    if (data[i][0] == id) return i + 1; // 1-indexed row number
  }
  return -1;
}

function deleteRowById(sheet, id) {
  var rowIndex = findRowById(sheet, id);
  if (rowIndex > 0) sheet.deleteRow(rowIndex);
}

function fetchAllPaginated(fetchFn) {
  var all = [];
```

```
  var page = 1;
  while (true) {
    var result = fetchFn(page);
    if (result.data) all = all.concat(result.data);
    if (!result.hasMore && !result.continuationToken) break;
    if (result.data && result.data.length === 0) break;
    page++;
    Utilities.sleep(100); // Rate limiting
  }
  return all;
}
```

Currency Formatting (ClientUtils.html)

```
function formatCurrency(amount) {
  return "$" + parseFloat(amount || 0).toFixed(2).replace(/\d(?=(\d{3})+\.)/g, "$&
}

function formatPercent(value) {
  return parseFloat(value || 0).toFixed(2) + "%";
}

function parseNumber(str) {
  return parseFloat(str) || 0;
}
```

Form Handling Pattern

```
function collectFormData(formId) {
  var form = document.getElementById(formId);
  var inputs = form.querySelectorAll("input, select, textarea");
  var data = {};
  inputs.forEach(function(input) {
    if (input.name) {
      if (input.type === "checkbox") {
        data[input.name] = input.checked;
      } else {
        data[input.name] = input.value;
      }
    }
  });
  return data;
}

function populateForm(data) {
  Object.keys(data).forEach(function(key) {
    var el = document.getElementById(key);
```

```
    if (el) {
      if (el.type === "checkbox") {
        el.checked = data[key];
      } else {
        el.value = data[key];
      }
    }
  });
}
```

Confirmation Dialog Pattern

```
<div id="confirmDialog" class="modal" style="display:none;">
  <div class="modal-content">
    <p id="confirmMessage"></p>
    <div class="modal-actions">
      <button onclick="confirmAction()" class="btn btn-danger">Confirm</button>
      <button onclick="closeConfirm()" class="btn btn-outline">Cancel</button>
    </div>
  </div>
</div>
<script>
var pendingAction = null;
function showConfirm(message, action) {
  document.getElementById("confirmMessage").textContent = message;
  pendingAction = action;
  document.getElementById("confirmDialog").style.display = "flex";
}
function confirmAction() {
  if (pendingAction) pendingAction();
  closeConfirm();
}
function closeConfirm() {
  document.getElementById("confirmDialog").style.display = "none";
  pendingAction = null;
}
</script>
```

Summary

This document provides a complete blueprint for rebuilding PricebookCalc in Google Workspace. The key mapping is:

- **Google Sheets** replaces PostgreSQL (5 sheet tabs = 5 database tables)
- **Apps Script** replaces both Express.js backend and React frontend
- **HTML Service** provides the web UI with server-side templating

- **google.script.run** replaces REST API calls
- **CacheService** replaces in-memory OAuth token caching
- **Script Properties** replaces environment variables
- **Client-side localStorage** works identically for per-device settings

All calculation formulas, business logic, and feature behaviors documented here are taken directly from the original codebase and are ready to implement.